

User Guide: Compass-Needle Lattice Simulator

1ex]Practicalreferenceforcompass.py/compassV02.pyV2.0.2

J. P. Sinnecker

Guide version: July 2026, aligned with Report V52

Source code timestamp: 2026-07-10T11:50:24-03:00

Abstract

This guide describes the current research branch of the compass-needle lattice simulator, distributed as `compass.py` or `compassV02.py`. The code simulates two-dimensional arrays of rigid compass needles as fixed-center point magnetic dipoles with continuous in-plane angular dynamics. Version V2.0.2 emphasizes reproducible numerical experiments: explicit physical parameters, explicit cutoff conventions, timestamped metadata, CPU/GPU backends, native research observables, avalanche counters, energy accounting, First-Order Reversal Curve protocols, demagnetization protocols, initial and final state snapshots, and automatic lattice PNG figures. This document replaces the older V78 user guide and is consistent with the updated project report V52 and the latest uploaded code.

Contents

1 What changed relative to the older user guide

The earlier user guide described the V78/V79 family of the simulator. The current code is a research-grade V2.0.2 branch. The most important changes are:

1. **File identity and reproducibility.** The source file contains a timestamp, stored in the constant `SOURCE_FILE_TIMESTAMP`. Each run writes this value to the JSON metadata together with `created_datetime_utc` and `created_datetime_local`.
2. **Output tree.** Each run writes `data/`, `meta/`, `states/`, and `images/` subdirectories under `-out_dir`.
3. **Initial and final states.** The code now saves both `states/<tag>_initial.npz` and `states/<tag>_final.npz`. The initial snapshot is saved before time integration, after lattice generation, physical-parameter derivation, field-protocol construction, tensor precomputation, and metadata assembly.
4. **Automatic lattice PNGs.** Every run automatically generates `images/<tag>_initial_lattice.png`, `images/<tag>_final_lattice.png`, and `images/<tag>_initial_final.png`. The option `-make_plot` is now only for additional CSV diagnostic plots.
5. **New plotting controls.** Automatic lattice figures are controlled by `-png_dpi`, `-png_transparent`, `-png_with_axes`, and `-png_no_panel_titles`.
6. **Research observables.** The CSV now contains magnetization components, polar and nematic order parameters, director angle, square-axis population, activity counters, energy terms, and angular-velocity diagnostics.
7. **Explicit cutoff convention.** `-cutoff_shells` gives a cutoff in units of the measured nearest-neighbour distance r_{nn} ; `-cutoff_m` gives an absolute cutoff in metres and overrides `-cutoff_shells`.
8. **Field protocols.** Supported protocols are `static`, `hysteresis`, `forc`, `sine`, `step_up`, `step_pos`, `step_down`, `step_neg`, `pulse`, `demag_rot`, and `demag_linear`.

2 Quick start

Assume the current source file has been saved locally as `compass.py`. If the downloaded file has another name, for example `compass(3).py` or `compassV02.py`, rename or copy it:

```
cp compassV02.py compass.py
# or, if your browser saved the uploaded file as compass(3).py:
cp 'compass(3).py' compass.py
```

Check that the local file is the current branch:

```
python3 compass.py --help | grep -E "png|state|out_dir|tag|verbose"
```

A current V2.0.2 file should show the output options `-out_dir`, `-tag`, `-verbose`, and the PNG options `-png_dpi`, `-png_transparent`, `-png_with_axes`, and `-png_no_panel_titles`.

Run a short smoke test:

```
rm -rf v02_png_test

python3 compass.py \
  --field_mode static \
  --geometry square \
  --N 3 --M 3 \
```

```
--t_sim 0.005 \
--out_dir v02_png_test \
--tag static_quick \
--verbose
```

Expected files:

```
v02_png_test/data/static_quick.csv
v02_png_test/meta/static_quick.json
v02_png_test/states/static_quick_initial.npz
v02_png_test/states/static_quick_final.npz
v02_png_test/images/static_quick_initial_lattice.png
v02_png_test/images/static_quick_final_lattice.png
v02_png_test/images/static_quick_initial_final.png
```

3 Installation and dependencies

The simulator is a single Python script. The required runtime dependencies are minimal:

- Python 3.10 or later is recommended.
- `numpy` is required.
- `matplotlib` is required for automatic PNG generation and optional quick plots.
- `cupy` is optional and only needed for GPU execution with `-use_gpu`.

A typical CPU-only environment can be prepared with:

```
python3 -m pip install numpy matplotlib
```

If using Anaconda or Miniconda:

```
conda create -n compass-v02 python=3.11 numpy matplotlib
conda activate compass-v02
```

GPU support requires a CuPy build compatible with the installed CUDA version. If `-use_gpu` is requested but CuPy is unavailable or fails to initialize, the code falls back to CPU/NumPy and prints a warning.

4 Physical model

The simulator treats each compass needle as a rigid point magnetic dipole located at a fixed pivot and free to rotate in the sample plane. For needle i the equation of motion is

$$I\ddot{\theta}_i = \tau_i^{\text{dip}} + \tau_i^{\text{ext}} - b\dot{\theta}_i, \quad (1)$$

where I is the moment of inertia, b is the viscous damping coefficient, θ_i is the in-plane angular coordinate, τ_i^{dip} is the dipolar torque from all other needles within the cutoff, and τ_i^{ext} is the torque from the external field.

The magnetic moment of each needle is

$$\mathbf{m}_i = m(\cos \theta_i, \sin \theta_i), \quad (2)$$

and the dipolar field at site i is computed from

$$\mathbf{B}_i^{\text{dip}} = \frac{\mu_0}{4\pi} \sum_{j \neq i} \left[\frac{3(\mathbf{m}_j \cdot \hat{\mathbf{r}}_{ij})\hat{\mathbf{r}}_{ij} - \mathbf{m}_j}{r_{ij}^3} \right]. \quad (3)$$

The torque is the out-of-plane component of $\mathbf{m}_i \times \mathbf{B}_i$:

$$\tau_i = m_{x,i}(B_{y,i}^{\text{dip}} + B_y^{\text{ext}}) - m_{y,i}(B_{x,i}^{\text{dip}} + B_x^{\text{ext}}). \quad (4)$$

4.1 Physical default dimensions

The current code separates the lattice spacing from the physical needle size. This is an important correction relative to older visualization-oriented versions. The default constants are:

Quantity	Code symbol	Default value
Centre-to-centre pivot spacing	CENTER_DISTANCE_DEFAULT	13 mm
Half spacing used internally	R_DEFAULT	6.5 mm
Needle blade length	NEEDLE_LEN_DEFAULT	10 mm
Needle blade width	NEEDLE_WIDTH_DEFAULT	3 mm
Needle thickness	NEEDLE_THICKNESS_DEFAULT	0.4 mm
Steel density	STEEL_DENSITY_DEFAULT	7850 kg m ⁻³
Steel saturation magnetization	STEEL_MS_SATURATION_DEFAULT	1.59 × 10 ⁶ A m ⁻¹
Damping coefficient	DAMPING_DEFAULT	5 × 10 ⁻⁸ N m s rad ⁻¹

The magnetic steel blade is approximated as a rhombus of area $A = LW/2$. The magnetic moment is estimated as

$$m = M_s t \left(\frac{LW}{2} - \pi r_p^2 \right), \quad (5)$$

where r_p is an optional pivot-hole radius. The moment of inertia is estimated for a thin rhombus-shaped lamina, with optional subtraction of the pivot hole and optional addition of a central pivot mass.

4.2 Reference field and natural time scale

The code defines a reference dipolar field using the actual nearest-neighbour distance r_{nn} measured from the generated lattice:

$$B_{\text{ref}} = \frac{\mu_0}{4\pi} \frac{2m}{r_{\text{nn}}^3}. \quad (6)$$

If `-B_ext` is not supplied, the applied field amplitude is set to

$$B_{\text{ext}} = \text{B_max_factor} B_{\text{ref}}, \quad (7)$$

with default `B_max_factor=8.0`. The natural angular frequency used to set the timestep is

$$\omega_0 = \sqrt{\frac{m B_{\text{eff}}}{I}}, \quad B_{\text{eff}} = \max(|B_{\text{ext}}|, B_{\text{ref}}), \quad (8)$$

and

$$\Delta t = \text{dt_factor} \frac{2\pi}{\omega_0}. \quad (9)$$

The default value is `-dt_factor 0.04`. The damping regime is summarized by

$$Q = \frac{\omega_0 I}{b}. \quad (10)$$

5 Lattice geometries

The option `-geometry` selects the spatial arrangement of fixed pivots. The supported choices are `square`, `triangular`, and `honeycomb`.

Table 1: Lattice geometries supported by the current code.

Geometry	Description
<code>square</code>	$N \times M$ rectangular grid with nearest neighbours along orthogonal directions. Bulk coordination is four. This is the natural baseline for domains, hysteresis loops, and avalanche counting.
<code>triangular</code>	Offset-row triangular lattice with higher local coordination and stronger geometrical frustration. Bulk coordination is six for large arrays away from boundaries.
<code>honeycomb</code>	Honeycomb-like finite grid with hexagonal holes and lower coordination. The nominal size is controlled by <code>-N</code> and <code>-M</code> ; the actual number of pivots K is written to metadata because finite masks and boundaries determine the final site count.

For all geometries the code measures r_{nn} directly from the generated coordinates and stores it in both the state files and metadata. This is more reliable than assuming a value from nominal lattice parameters.

6 Dipolar tensor, cutoff, and boundary conditions

For efficiency the code precomputes three interaction matrices, `Axx`, `Axy`, and `Ayy`, such that the dipolar field can be evaluated by matrix-vector products:

$$B_x^{\text{dip}} = A_{xx}m_x + A_{xy}m_y, \quad (11)$$

$$B_y^{\text{dip}} = A_{xy}m_x + A_{yy}m_y. \quad (12)$$

The cutoff can be specified in two ways:

- `-cutoff_shells <value>` specifies the cutoff as a multiple of the measured nearest-neighbour distance r_{nn} . The default is 3.5.
- `-cutoff_m <value>` specifies the absolute cutoff in metres. If supplied, it overrides `-cutoff_shells`.

The metadata stores both the physical cutoff in metres and the equivalent number of nearest-neighbour shells, together with a `cutoff_policy` field indicating which convention was used.

Periodic boundary conditions are activated with `-pbc`. The number of replicated images in each direction is controlled by `-n_images`. For finite experimental compass arrays, free boundaries are often the physically relevant default; PBC should be used as a controlled modelling choice.

The approximate memory required by the tensor is

$$3K^2 \times \text{sizeof}(\text{dtype}), \quad (13)$$

where K is the number of needles. The option `-tensor_mem_limit_gb` prevents accidental construction of tensors larger than the requested limit. `-float32` can reduce memory use, but `float64` remains preferable for final data whenever feasible.

7 Field protocols

The external field is described by a scalar protocol $B(t)$ and a direction angle `-phi_ext_deg`. Except for `demag_rot`, the vector field is

$$\mathbf{B}(t) = B(t)(\cos \phi, \sin \phi). \quad (14)$$

Table 2: Field protocols supported by `compass.py` / `compassV02.py`.

Mode	Meaning	Main options
<code>static</code>	Constant field for <code>-t_sim</code> .	<code>-B_ext</code> , <code>-B_max_factor</code> , <code>-phi_ext_deg</code>
<code>hysteresis</code>	Five-branch major loop: $0 \rightarrow +B \rightarrow 0 \rightarrow -B \rightarrow 0 \rightarrow +B$.	<code>-t_sim</code> , <code>-hyst_spacing</code> , <code>-hyst_log_k</code>
<code>forc</code>	Sequence of First-Order Reversal Curves: saturation, ramp down to reversal field B_r , ramp up to $B_{\max}$.	<code>-forc_n_curves</code> , <code>-forc_Br_min</code> , <code>-forc_t_sat</code> , ramps or rate
<code>sine</code>	Sinusoidal field $B_{\max} \sin(2\pi ft)$.	<code>-field_freq</code> , <code>-t_sim</code>
<code>step_up</code>	/ Field is zero before <code>-field_delay</code> and equals $B_{\max}$ after the delay.	<code>-field_delay</code> , <code>-t_sim</code>
<code>step_pos</code>		
<code>step_down</code>	/ Field equals $B_{\max}$ before <code>-field_delay</code> and is zero afterward.	<code>-field_delay</code> , <code>-t_sim</code>
<code>step_neg</code>		
<code>pulse</code>	Field is zero, then on for <code>-t_pulse</code> , then zero again.	<code>-field_delay</code> , <code>-t_pulse</code> , <code>-t_sim</code>
<code>demag_rot</code>	Rotating field with linearly decaying envelope, followed by relaxation.	<code>-demag_freq</code> , <code>-demag_cycles</code> , <code>-t_relax_after</code>
<code>demag_linear</code>	Fixed-direction field with linearly decaying amplitude, followed by relaxation.	<code>-demag_freq</code> , <code>-demag_cycles</code> , <code>-t_relax_after</code>

For `forc` and demagnetization modes, the effective simulation time is computed internally from the protocol parameters. For all other modes, `-t_sim` sets the total duration.

8 Output files and data products

Every run creates a structured output directory:

```
<out_dir>/
  data/
    <tag>.csv
  meta/
    <tag>.json
  states/
    <tag>_initial.npz
    <tag>_final.npz
  images/
```

```
<tag>_initial_lattice.png
<tag>_final_lattice.png
<tag>_initial_final.png
```

If `-tag` is omitted, the code builds a tag from geometry, field mode, lattice size, and seed. For reproducible campaigns, it is better to supply explicit tags.

8.1 CSV time series

The CSV file is the main time-series output. The current header is:

```
step,t_s,Bx_T,By_T,B_scalar_T,branch,forc_index,
Mx,My,M_proj,S1,S2,theta_director_rad,q_axis,
flip_field,flip_angle,E_dip_J,E_ext_J,E_kin_J,E_total_J,
omega_rms_rad_s,omega_max_rad_s
```

Table 3: Meaning of the current CSV columns.

Column	Meaning
<code>step, t_s</code>	Integration step and physical time in seconds.
<code>Bx_T, By_T, B_scalar_T</code>	Field components and scalar protocol value in tesla.
<code>branch, forc_index</code>	Protocol branch label and FORC curve index. <code>forc_index=-1</code> outside FORC.
<code>Mx, My, M_proj</code>	Mean polar magnetization components and projection along the drive axis.
<code>S1</code>	Polar order parameter $ \langle e^{i\theta} \rangle $.
<code>S2</code>	Nematic/director order parameter $ \langle e^{2i\theta} \rangle $. Useful when antiparallel needles cancel polar magnetization but remain aligned as axes.
<code>theta_director_rad</code>	Director angle $\frac{1}{2} \arg \langle e^{2i\theta} \rangle$.
<code>q_axis</code>	Population imbalance between x -like and y -like axes. Mainly meaningful for square-type director analysis.
<code>flip_field</code>	Count of sign changes relative to the drive axis accumulated since the previous CSV row.
<code>flip_angle</code>	Count of rotations larger than <code>-flip_angle_deg</code> since the previous CSV row.
<code>E_dip_J, E_ext_J, E_kin_J, E_total_J</code>	Dipolar, external, kinetic, and total energy in joules.
<code>omega_rms_rad_s, omega_max_rad_s</code>	Root-mean-square and maximum angular velocity.

8.2 JSON metadata

The JSON file records the exact run configuration and derived physical quantities. Important fields include:

- `program`, `version`, and `source_file_timestamp`;
- `created_unix_time`, `created_datetime_utc`, and `created_datetime_local`;
- `config`: the full effective simulation configuration;
- `derived`: K , r_{nn} , L_x , L_y , needle size, B_{ref} , B_{eff} , ω_0 , T_0 , Δt , number of steps, Q , cutoff policy, tensor memory, and dtype;

- `domain_statistics_final`: connected-component statistics of final polar domains;
- `runtime_s`: wall-clock runtime.

8.3 NPZ state files

The initial and final NPZ files contain:

```
xs, ys, theta, omega, r_nn, metadata_json
```

These files are sufficient to regenerate lattice images without rerunning the simulation. The state files are also the correct starting point for custom domain, correlation, or spatial Fourier analysis.

8.4 Automatic lattice PNGs

The code always generates three lattice figures:

- `<tag>_initial_lattice.png`: clean lattice drawing of the initial state;
- `<tag>_final_lattice.png`: clean lattice drawing of the final state;
- `<tag>_initial_final.png`: side-by-side comparison.

The figure style follows the project convention: a blue half for one compass pole, a white half for the opposite pole, and a small grey central pivot. The options are:

Option	Effect
<code>-png_dpi</code>	Resolution of automatic PNG files; default 300.
<code>-png_transparent</code>	Save transparent background.
<code>-png_with_axes</code>	Include axes, grid, labels, and titles.
<code>-png_no_panel_titles</code>	Remove Initial/Final titles from the side-by-side panel.

The option `-make_plot` is independent from these lattice images. When supplied, it adds CSV-based quicklook plots such as an $M(B)$ curve and an order-parameter plot.

9 Command-line reference

This section groups the most important command-line parameters. Use `python3 compass.py -help` for the complete live reference from the installed file.

Table 4: Main command-line options.

Option(s)	Meaning
<code>-geometry</code>	Choose <code>square</code> , <code>triangular</code> , or <code>honeycomb</code> .
<code>-N</code> , <code>-M</code>	Nominal number of rows and columns. For honeycomb, K is determined by the finite mask and should be read from metadata.
<code>-R</code>	Half the centre-to-centre pivot spacing. Default 6.5 mm.
<code>-needle_len</code> , <code>-needle_width</code> , <code>-needle_thickness</code>	Physical blade dimensions.

Option(s)	Meaning
-use_legacy_size_from_R	If set to 1, uses the old visualization convention in which needle length follows <code>needle_frac*2R</code> . The research default is 0.
-moment, -inertia	Directly override magnetic moment or moment of inertia.
-steel_density, -steel_Ms, -steel_Bsat	Material parameters used when deriving m and I . <code>-steel_Bsat</code> overrides <code>-steel_Ms</code> .
-pivot_radius, -pivot_thickness, -pivot_density, -pivot_mass	Optional pivot/hole corrections to inertia and magnetic moment.
-damping, -damping_noise	Uniform damping and optional relative random damping variation per needle.
-t_sim, -dt_factor, -noise, -seed, -log_every	Simulation time, timestep factor, initial angular noise, random seed, and CSV logging cadence.
-cutoff_shells, -cutoff_m	Dipolar cutoff convention. Absolute cutoff overrides shell cutoff.
-pbc, -n_images	Periodic boundary conditions and finite image count.
-tensor_mem_limit_gb, -float32	Memory protection and optional single precision.
-field_mode	Select field protocol.
-B_ext, -B_max_factor, -phi_ext_deg	Field amplitude and direction.
-field_freq, -field_delay, -t_pulse	Parameters for sine, step, and pulse protocols.
-hyst_spacing, -hyst_log_k	Hysteresis branch spacing.
-forc_Br_min, -forc_n_curves, -forc_t_sat, -forc_t_ramp_down, -forc_t_ramp_up, -forc_rate	FORC protocol controls.
-demag_freq, -demag_cycles, -t_relax_after	Demagnetization protocol controls.
-out_dir, -tag	Output root and run label.
-use_gpu, -progress, -verbose	Execution diagnostics.
-make_plot	Add CSV diagnostic quick plots. Does not control lattice PNGs.
-png_dpi, -png_transparent, -png_with_axes, -png_no_panel_titles	Controls automatic lattice PNGs.
-domain_tol_deg	Angular tolerance used for final polar-domain connected-component statistics.

10 Recommended test commands

10.1 Batch test of the ordinary field modes

```
mkdir -p field_mode_tests

for mode in static hysteresis sine step_up step_down pulse demag_rot demag_linear
do
    python3 compass.py \
        --field_mode "$mode" \
        --geometry square \
        --N 5 --M 5 \
        --t_sim 0.20 \
        --field_delay 0.05 \
        --t_pulse 0.05 \
        --demag_cycles 3 \
        --demag_freq 2.0 \
        --t_relax_after 0.20 \
        --noise 0.5 \
        --seed 1 \
        --out_dir "field_mode_tests/$mode" \
        --tag "${mode}_quick" \
        --verbose
done
```

Each run should create initial and final NPZ files and the three automatic lattice PNG files. For example, the static run should create:

```
field_mode_tests/static/images/static_quick_initial_lattice.png
field_mode_tests/static/images/static_quick_final_lattice.png
field_mode_tests/static/images/static_quick_initial_final.png
```

10.2 FORC smoke test

FORC runs should be tested separately because their total duration is constructed from the FORC schedule rather than directly from `-t_sim`.

```
python3 compass.py \
    --field_mode forc \
    --geometry square \
    --N 5 --M 5 \
    --forc_n_curves 4 \
    --forc_t_sat 0.01 \
    --forc_t_ramp_down 0.02 \
    --forc_t_ramp_up 0.03 \
    --noise 0.5 \
    --seed 1 \
    --out_dir field_mode_tests/forc_quick \
    --tag forc_quick \
    --verbose
```

10.3 Geometry smoke tests

```
for geom in square triangular honeycomb
do
    python3 compass.py \
        --field_mode static \
```

```

--geometry "$geom" \
--N 6 --M 6 \
--t_sim 0.02 \
--noise 0.5 \
--seed 3 \
--out_dir "geometry_tests/$geom" \
--tag "${geom}_static" \
--verbose
done

```

Inspect the corresponding `images/` directories to confirm that the lattice positions and initial/final orientations are plausible.

11 Example research runs

11.1 Hysteresis loop with automatic figures

```

python3 compass.py \
--field_mode hysteresis \
--geometry square \
--N 20 --M 20 \
--t_sim 2.0 \
--noise 1.5 \
--seed 11 \
--out_dir results/square_hysteresis \
--tag square_N20_hyst_seed11 \
--make_plot \
--verbose

```

This produces the standard CSV, metadata, initial/final NPZ state files, the three lattice PNGs, and optional CSV quick plots.

11.2 Damping comparison

```

for damping in 1e-9 3e-9 1e-8 3e-8 1e-7
do
python3 compass.py \
--field_mode hysteresis \
--geometry square \
--N 16 --M 16 \
--t_sim 2.0 \
--damping "$damping" \
--noise 1.5 \
--seed 21 \
--out_dir "results/damping_$damping" \
--tag "square_hyst_b_${damping}" \
--verbose
done

```

Use the JSON metadata to read the effective Q for each run, rather than inferring it manually.

11.3 Rotating demagnetization followed by relaxation

```

python3 compass.py \
--field_mode demag_rot \
--geometry triangular \

```

```
--N 18 --M 18 \
--demag_freq 2.0 \
--demag_cycles 10 \
--t_relax_after 1.0 \
--noise 1.5 \
--seed 9 \
--out_dir results/triangular_demag_rot \
--tag tri_N18_demag_rot_seed9 \
--verbose
```

11.4 Transparent PNGs for figures or slides

```
python3 compass.py \
--field_mode static \
--geometry honeycomb \
--N 8 --M 8 \
--t_sim 0.02 \
--noise 0.7 \
--seed 4 \
--out_dir results/honeycomb_static \
--tag honeycomb_static_transparent \
--png_transparent \
--png_no_panel_titles \
--verbose
```

12 Reading results in Python

A minimal reader for the CSV and metadata is:

```
from pathlib import Path
import json
import numpy as np

run = Path("results/square_hysteresis")
tag = "square_N20_hyst_seed11"

csv = np.genfromtxt(run / "data" / f"{tag}.csv",
                    delimiter=",", names=True, dtype=None, encoding=None)

with open(run / "meta" / f"{tag}.json") as f:
    meta = json.load(f)

print("K =", meta["derived"]["K"])
print("Q =", meta["derived"]["Q"])
print("B_ref =", meta["derived"]["B_ref_T"], "T")
print("source timestamp =", meta["source_file_timestamp"])

B_mT = csv["B_scalar_T"] * 1e3
M = csv["M_proj"]
S2 = csv["S2"]
flips = csv["flip_field"]
```

A minimal reader for a saved state is:

```
import numpy as np
import json
```

```
state = np.load("results/square_hysteresis/states/square_N20_hyst_seed11_final.npz")
xs = state["xs"]
ys = state["ys"]
theta = state["theta"]
omega = state["omega"]
r_nn = float(state["r_nn"])
metadata = json.loads(str(state["metadata_json"]))
```

13 Troubleshooting

13.1 The run creates data/, meta/, and states/, but no images/

This usually means the local file is not the current V2.0.2 code. Verify:

```
python3 compass.py --help | grep -E "png|state|out_dir|tag|verbose"
grep -n "initial_lattice\\|final_lattice\\|initial_final\\|png_dpi" compass.py
```

If the PNG options are missing, replace the local file with the latest `compass.py` / `compassV02.py`.

13.2 A command reports unrecognized arguments

Check that the file being executed is the current file:

```
which python3
pwd
ls -al compass.py compassV02.py
python3 compass.py --help | head -40
```

The most common cause is running an older `compass.py` while editing or downloading `compassV02.py` in another directory.

13.3 The run is too slow or uses too much memory

Reduce `-N` and `-M`, lower `-cutoff_shells`, use `-float32` for exploratory runs, or reduce the number of periodic images. For final data, prefer `float64` if memory allows.

13.4 The FORC run takes much longer than expected

The total FORC duration is approximately the sum over curves of saturation time, ramp-down time, and ramp-up time. Reduce `-forc_n_curves` for a smoke test, then increase it for production.

13.5 The lattice images look cropped or too small

Use `-png_with_axes` for debugging. The clean default removes axes and uses a tight bounding box, which is best for reports but can look visually different from plotting-window output.

14 Recommended workflow

For a reproducible research campaign, use the following workflow:

1. Verify the code version with `-help` and by checking `SOURCE_FILE_TIMESTAMP`.
2. Run the static smoke test and confirm that `images/` is created.

3. Run one small test for each field mode.
4. Inspect the JSON metadata for K , r_{nn} , B_{ref} , Q , cutoff policy, and runtime.
5. For each production parameter set, store the complete output directory, not only the CSV file.
6. Analyze magnetization, S_1 , S_2 , flip counters, and energies together. Avoid interpreting a single observable as the full state of the system.
7. Use multiple seeds for any claim about avalanche statistics or domain distributions.
8. Treat PBC, cutoff, damping, and field protocol as part of the scientific definition of the experiment and report them explicitly.

15 Scientific caveat

The simulator is a continuous-angle inertial dipolar model. It is not a literal micromagnetic model of nanolithographic artificial spin ice. It does not include shape-anisotropy switching astroids, intrinsic switching-field distributions, domain-wall reversal, material-specific Gilbert damping, or finite-size micromagnetic texture inside each island. Its value is that it isolates what follows from geometry, long-range anisotropic dipolar coupling, inertia, damping, and field history alone. That makes it especially useful as a clean reference model for interpreting hysteresis, FORC structure, avalanches, relaxation, and director memory in mechanically realizable compass-needle arrays.

A Complete minimal command set

A.1 Static

```
python3 compass.py --field_mode static --geometry square --N 6 --M 6 \
  --t_sim 0.10 --noise 0.5 --seed 1 \
  --out_dir tests/static --tag static_test --verbose
```

A.2 Hysteresis

```
python3 compass.py --field_mode hysteresis --geometry square --N 8 --M 8 \
  --t_sim 0.50 --noise 1.5 --seed 2 \
  --out_dir tests/hysteresis --tag hysteresis_test --verbose
```

A.3 Sinusoidal field

```
python3 compass.py --field_mode sine --geometry square --N 6 --M 6 \
  --t_sim 1.00 --field_freq 2.0 --noise 0.5 --seed 3 \
  --out_dir tests/sine --tag sine_test --verbose
```

A.4 Pulse

```
python3 compass.py --field_mode pulse --geometry square --N 6 --M 6 \
  --t_sim 0.60 --field_delay 0.10 --t_pulse 0.20 \
  --noise 1.0 --seed 4 \
  --out_dir tests/pulse --tag pulse_test --verbose
```

A.5 FORC

```
python3 compass.py --field_mode forc --geometry square --N 6 --M 6 \  
  --forc_n_curves 5 --forc_t_sat 0.02 \  
  --forc_t_ramp_down 0.04 --forc_t_ramp_up 0.06 \  
  --noise 1.0 --seed 5 \  
  --out_dir tests/forc --tag forc_test --verbose
```

A.6 Rotating demagnetization

```
python3 compass.py --field_mode demag_rot --geometry square --N 6 --M 6 \  
  --demag_freq 2.0 --demag_cycles 5 --t_relax_after 0.50 \  
  --noise 1.5 --seed 6 \  
  --out_dir tests/demag_rot --tag demag_rot_test --verbose
```

B Version log

Version	Date	Notes
V78/V79 guide family	June–July 2026	Older visual-simulator documentation with field modes and campaign examples, but before the V2 research-output refactoring.
V2.0.2 code branch	10 July 2026	Explicit cutoff conventions, physical needle dimensions independent of lattice spacing, research observables, activity counters, energy accounting, CPU/GPU tensor backend, timestamped metadata, initial and final state files, and automatic lattice PNG figures.
Report V52	10 July 2026	Report updated to match <code>compass(3).py</code> / <code>compassV02.py</code> , including CSV header, metadata fields, output directory structure, and automatic PNG behavior.
This user guide	July 2026	User-facing operational guide updated to match the newest report and the current <code>compass.py</code> workflow.